

1.

a) Napisati MPI program koji realizuje množenje matrice $A_{n \times k}$ i matrice $B_{k \times m}$, čime se dobija i prikazuje rezultujuća matrica C . Takođe, program pronalazi i prikazuje minimum elemenata svake vrste matrice A . Izračunavanje se obavlja tako što master proces šalje svakom procesu po k/p kolona matrice A i po k/p vrsta matrice B (p - broj procesa, k deljivo sa p). Master proces inicijalizuje matricu A i matricu B . Predvideti da se slanje svih kolona matrice A jednom procesu obavlja odjednom i direktno iz matrice A . Predvideti da se slanje svih vrsta matrice B jednom procesu obavlja odjednom i direktno iz matrice B . Svi procesi učestvuju u izračunavanjima potrebnim za generisanje rezultata programa. Rezultati programa se nalaze u proizvoljnom procesu koji ih i prikazuje. Zadatak realizovati korišćenjem grupnih operacija.

b) Napisati MPI program koji će omogućiti komunikaciju samo procesa na sporednoj dijagonali procesa u kvadratnoj mreži procesa i to onih čiji je rank manji od zadate vrednosti x . Obezbediti da svaki proces ovog komunikatora dobije po jednu vrstu matrice A . Prikazati identifikatore procesa koji pripadaju novom komunikatoru, kao i maksimum svih proizvoda primljenih vrsta matrice A . Prikazati i identifikator procesa koji sadrži taj maksimum u novom komunikatoru.

2. Na programskom jeziku C/C++, korišćenjem CUDA tehnologije, napisati program koji u nizu A , dužine n , koji predstavlja RNK sekvencu, pronalazi traženu podsekvencu, dužine m . Svaka RNK sekvenca i podsekvencu se sastoje od nukleotida (A, U, C, G). Rezultat upisati u niz B , koji na početku svake pronađene sekvence upisuje 1, a na ostalim pozicijama 0. Obratiti pažnju na efikasnost paralelizacije. Omogućiti pozivanje kernela za matrice proizvoljne veličine.

3. Data je petlja:

```
int sep = 1;
for ( i=0; i<N-1; i++ )
{
    a[i] = a[i+1] + b[i] * c[offset+N-i-1];
    sep = sep * t[i+1];
}
```

a) Da li je moguće izvršiti paralelizaciju date petlje bez modifikacija? Obrazložiti. Ukoliko postoje zavisnosti, navesti koje su i objasniti ih.

b) Transformisati petlju tako da paralelizacija bude moguća i paralelizaciju izvršiti korišćenjem OpenMP.

c) Razvijanjem petlje za konkretne inicijalizacije vrednosti nizova a, b, c i t , testirati sekvencijalno i paralelno rešenje i pokazati da je petlja korektno paralelizovana rešenjem pod b).

Napomena: zadaci koji daju odgovor samo na stavku b) neće biti bodovani.

4. Program ima 40% dela koji se ne može paralelizovati, dok ostatak programa može biti podeljen na 6 jednakih delova koji se mogu izvršavati paralelno. Koliko se maksimalno ubrzanje može ostvariti kod izvršenja ovog programa na 4-jezgarnom procesoru? Koliko je max ubrzanje ako imamo 10 procesora na raspolaganju?

5. Da li se sledeća petlja može vektorizovati:

```
for (i = 1; i<N; i++)
    A(i) = A(N-i+1) + C;
```

Ako je odgovor negativan, transformisati petlju tako da se može vektorizovati.

6. Sledeći kod se izvršava na dvo procesorskom sistemu. Za održavanje keš koherencije koristi se write-one protokol:

- (1) LW x1, 0(x5)
- (2) LW x2, 0(x6)
- (3) SW x3, 0(x6)
- (4) SW x2, 0(x5)
- (5) LW x1, 0(x6)

Adrese x5 i x6 se ne preslikavaju u isti keš blok. Neka se instrukcije (1) do (5) izvršavaju od strane procesora A i B u sledećem redosledu: A.1, A.2, B.1, B.2, A.3, B.3, B.4, A.4, A.5, B.5 (A i B su procesori). Popuniti donju tabelu ako se za održavanje keš koherencije koristi write-once protokol.

Procesor/ Instrukcija	Stanje x5 keš linije u proc. A	Stanje x6 keš linije u proc. A	Komanda keš kontrolera A (ako nema staviti --)	Stanje x5 keš linije u proc. B	Stanje x6 keš linije u proc. B	Komanda keš kontrolera B (ako nema staviti --)
A: LW x1, 0(x5)						
A: LW x2, 0(x6)						
B: LW x1, 0(x5)						
B: LW x2, 0(x6)						
A: SW x3, 0(x6)						
B: SW x3, 0(x6)						
B: SW x2, 0(x5)						
A: SW x2, 0(x5)						
A: LW x1, 0(x6)						
B: LW x1, 0(x6)						

7. a) Zadati je sledeći OpenMP program:

```
pragma omp parallel for private(i) num_threads(4) schedule(static, 10)
  for (int i = 0; i < 100; i++)
    a[i] = i;
```

Koja nit će izvršiti 76. iteraciju? Koje iteracije će izvršiti nit 2?

b) Optimizovati sledeći kod uklanjanjem nepotrebnih barijera:

```
#pragma omp parallel private(i)
{
  #pragma omp for
  for(i=0; i<n; i++)
    a[i] += b[i];
  #pragma omp for
  for(i=0; i<n; i++)
    c[i] += d[i];
  #pragma omp barrier
  #pragma omp for reduction(+:sum)
  for(i=0; i<n; i++)
    sum += a[i] + c[i];
}
```

c) Šta znači OpenMP

- Open Multi-Processing
- Open Memory Parallelization
- Open Message Passing
- Open Multiprocessing Protocol

d) Koja OpenMP direktiva se koristi da se obezbedi da samo jedna nit izvrši neki blok naredbi:

- #pragma omp master
- #pragma omp barrier
- #pragma omp single
- #pragma omp critical